

Self-Healing Cloud Infrastructure on AWS

Project Pitch

1. Introduction & Problem Statement

Modern cloud applications are expected to be highly available, yet hardware failures, OS panics, and unexpected resource spikes (like CPU or memory exhaustion) are inevitable. Manually responding to these incidents leads to high MTTR (Mean Time To Recovery) and potential application downtime.

2. Project Objective

The objective of this project is to design and implement a **fully automated, multi-layered self-healing architecture** on AWS. It demonstrates how cloud-native observability and automation can detect, alert, and resolve different classes of failures without human intervention, all while remaining 100% within the AWS Free Tier limits.

3. Architecture & Self-Healing Loops

The system is built upon three core self-healing loops to address failures at different levels of the stack:

- **Loop 1: Service Recovery (Application Level)** CloudWatch alarms monitor CPU and Memory. If utilization exceeds the threshold (e.g., 85%), an alert is sent via SNS, which triggers an AWS Lambda function. The Lambda function uses AWS Systems Manager (SSM) to automatically restart the struggling service, restoring application health.
- **Loop 2: OS Recovery (Instance Level)** If the underlying operating system becomes unresponsive (Instance Status Check Failure), a CloudWatch alarm triggers an EC2 Auto-Reboot action to recover the OS automatically.
- **Loop 3: Hardware Recovery (Infrastructure Level)** In the event of underlying AWS hardware failure (System Status Check Failure), an EC2 Auto-Recovery action is triggered. This migrates the instance to healthy hardware while preserving its IP address, Instance ID, and attached EBS volumes.

4. Key Features & Technologies

- **Infrastructure as Code (IaC):** The entire stack is provisioned using Terraform, ensuring reproducibility, version control, and consistency.
- **Real-time Observability:** A custom CloudWatch Dashboard provides a single-pane-of-glass view for metrics (CPU, Memory), Alarm States, and Lambda execution metrics.
- **Audit & Compliance:** AWS EventBridge captures all EC2 state changes (Start/Stop/Reboot) and triggers a Lambda function to log lifecycle events into an Amazon S3 bucket with versioning enabled, ensuring a tamper-proof audit trail.
- **CloudOps Dashboard:** A React-based frontend deployed on the EC2 instance visualizes real-time health data and metrics.
- **Discord ChatOps Integration:** A custom webhooks integration that instantly notifies the engineering team in a Discord channel whenever the self-healing system triggers an automated recovery action.
- **Production-Grade Infrastructure-as-Code (IaC):** Configured Terraform Remote State with **Amazon S3** (versioned state storage) and **Amazon DynamoDB** (distributed state locking), ensuring

concurrency safety and CI/CD readiness.

- **Automated SRE Incident Post-Mortems:** Upon every recovery event, the system automatically calculates MTTD/MTTR reliability metrics, generates a Markdown Root Cause Analysis (RCA) report, stores it in S3, and links it in Discord.
- **Custom Composite Health Score Engine (0–100):** A fully custom-built Python algorithm that normalises CPU utilisation, Memory utilisation, Uptime Stability, Metric Volatility (standard deviation), and Healing State into a single weighted composite "Infrastructure Health Score" with letter grading (A–F). This feature does not exist in AWS — it was engineered from scratch.
- **Predictive Failure Detection (Linear Regression ML):** A custom Lambda function runs every 5 minutes, fetches the last 30 minutes of CPU & Memory time-series data, applies a least-squares linear regression algorithm to calculate the trend slope, R^2 coefficient, and extrapolates 10 minutes into the future. If a threshold breach is predicted, it sends a **Pre-Alert** to Discord *before* the CloudWatch alarm fires — reducing Mean Time to Detect (MTTD) to near-zero.

5. Cost & Efficiency (Free Tier Compliance)

The architecture is meticulously designed to incur zero costs by operating entirely within the AWS Free Tier:

- **Compute:** 1x t2.micro EC2 Instance (Well within the 750 hours/month limit)
- **Storage:** 8 GB gp3 EBS Volume (Limit is 30 GB/month) + Minimal S3 Storage
- **Automation:** AWS Lambda & SNS (Expected usage is < 100 requests/month, limit is 1 Million)

6. Conclusion

This project practically demonstrates the principles of **Site Reliability Engineering (SRE)** and **Cloud Operations**. It goes beyond a standard web deployment by engineering a resilient system capable of surviving and self-correcting common cloud failures, serving as a comprehensive showcase of cloud-native fault tolerance and automation.